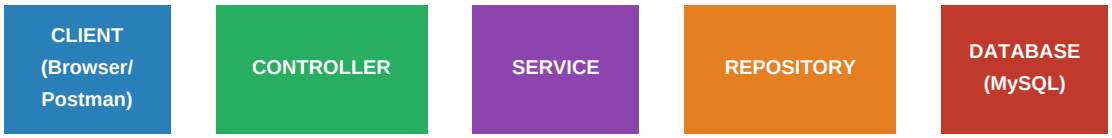
Ringkasan Materi Pelatihan

MATERI	DURASI
Java OOP	~2 jam
Overview Teknologi Spring Boot	~1 jam
Struktur Project Spring Boot & ORM	~3 jam
REST API Sederhana & Audit Trail	~3 jam
Security (JWT / Basic Auth)	~3 jam
File Transfer & Exception Handling	~3 jam
Testing dengan JUnit	~3 jam
Studi Kasus (Project)	~4 jam

■ *Dokumen ini adalah rangkuman yang telah DIREVISI dan diperlengkap. Gunakan sebagai panduan cepat selama pelatihan berlangsung.*

1. Gambaran Besar Teknologi

Ketika membuat aplikasi backend dengan Spring Boot, alur umumnya adalah:



Setiap lapisan memiliki tanggung jawab yang berbeda:

Lapisan	Tanggung Jawab	Anotasi Utama
Controller	Menerima HTTP request, mengembalikan response	@RestController, @GetMapping, ...
Service	Logika bisnis / business logic	@Service, @Transactional
Repository	Akses & manipulasi data ke database	@Repository, JpaRepository
Entity	Representasi tabel di database	@Entity, @Table, @Id
DTO	Objek data transfer request/response	Plain class / @Data (Lombok)

2. Java OOP (Object Oriented Programming)

2.1 Class & Object

Class = blueprint/cetakan. Object = hasil instansiasi class.

```
public class Mobil { String warna; String merk; int kecepatan; } // Membuat object Mobil mobilSaya = new Mobil(); mobilSaya.warna = "Merah";
```

2.2 Constructor

Constructor dijalankan otomatis saat object dibuat (new). Berguna untuk mengisi nilai awal.

```
public class User { String nama; int umur; // Constructor public User(String nama, int umur) { this.nama = nama; this.umur = umur; } } // Pemakaian User user = new User("Naila", 21);
```

2.3 Encapsulation

Data dibuat private, diakses lewat getter/setter. Tujuannya menjaga keamanan data dari akses luar.

```
public class User { private String nama; public String getNama() { return nama; } public void setNama(String nama) { this.nama = nama; } }
```

■ *Dengan Lombok @Data, getter/setter dibuat otomatis tanpa kode manual!*

2.4 Inheritance

Class anak mewarisi atribut dan method class induk. Gunakan keyword extends.

```
public class Hewan { public void makan() { System.out.println("Makan"); } } public class Kucing extends Hewan { public void mengeong() { System.out.println("Meong!"); } } // Kucing bisa memanggil makan() dari Hewan Kucing c = new Kucing(); c.makan(); c.mengeong();
```

2.5 Polymorphism

Method yang sama bisa memiliki perilaku berbeda di class turunan (method overriding).

```
public class Hewan { public void suara() { System.out.println("Suara hewan"); } } public class Kucing extends Hewan { @Override public void suara() { System.out.println("Meong"); } } public class Anjing extends Hewan { @Override public void suara() { System.out.println("Woof"); } }
```

2.6 Interface & Abstract Class

Interface = kontrak perilaku yang wajib diimplementasikan. Abstract Class = class yang tidak bisa diinstansiasi langsung.

```
public interface Kendaraan { void jalankan(); void berhenti(); } public class Mobil implements Kendaraan { @Override public void jalankan() { System.out.println("Mobil jalan"); } @Override public void berhenti() { System.out.println("Mobil berhenti"); } }
```

■ *Interface sering dipakai di Spring: Service interface → ServiceImpl class*

3. Struktur Project Spring Boot

Folder/File	Fungsi
controller/	Menerima request HTTP dari client
service/	Logika bisnis aplikasi
repository/	Query & akses database
entity/	Representasi tabel database (POJO + JPA)
dto/	Data Transfer Object – request & response body
config/	Konfigurasi (Security, CORS, Beans, dll)
exception/	Custom exception & global error handler
DemoApplication.java	Entry point – main class Spring Boot
application.properties	Konfigurasi datasource, port, JPA, dll

■ *Naming convention: ProductController, ProductService, ProductRepository, ProductEntity, ProductDTO*

4. REST API

HTTP Method	Endpoint Contoh	Fungsi	Anotasi Spring
GET	/products	Ambil semua data	@GetMapping
GET	/products/1	Ambil data by ID	@GetMapping("/{id}")
POST	/products	Tambah data baru	@PostMapping
PUT	/products/1	Update seluruh data	@PutMapping("/{id}")
PATCH	/products/1	Update sebagian data	@PatchMapping("/{id}")
DELETE	/products/1	Hapus data	@DeleteMapping("/{id}")

HTTP Status Code Penting

Kode	Status	Kapan Digunakan
200	OK	Request sukses (GET, PUT, DELETE)
201	Created	Data baru berhasil dibuat (POST)
204	No Content	Sukses tapi tidak ada data dikembalikan
400	Bad Request	Data request tidak valid
401	Unauthorized	Belum login / token tidak ada
403	Forbidden	Sudah login tapi tidak punya izin
404	Not Found	Data/endpoint tidak ditemukan
500	Internal Server Error	Error di sisi server

5. Controller – CRUD Lengkap

```
@RestController @RequestMapping("/products") public class ProductController { @Autowired private ProductService service; // GET semua produk @GetMapping public List getAll() { return service.getAll(); } // GET by ID @GetMapping("/{id}") public ResponseEntity getById(@PathVariable Long id) { return ResponseEntity.ok(service.getById(id)); } // POST – tambah produk baru @PostMapping public ResponseEntity create(@RequestBody ProductDTO dto) { return ResponseEntity.status(201).body(service.create(dto)); } // PUT – update produk @PutMapping("/{id}") public ResponseEntity update( @PathVariable Long id, @RequestBody ProductDTO dto) { return ResponseEntity.ok(service.update(id, dto)); } // DELETE – hapus produk @DeleteMapping("/{id}") public ResponseEntity delete(@PathVariable Long id) { service.delete(id); return ResponseEntity.noContent().build(); } }
```

Anotasi Penting di Controller

Anotasi	Fungsi
@RestController	Gabungan @Controller + @ResponseBody
@RequestMapping	Prefix path untuk semua method di class ini
@PathVariable	Ambil nilai dari path URL e.g. /products/{id}
@RequestBody	Ambil data dari body request (JSON)
@RequestParam	Ambil query string e.g. ?page=1&size=10
ResponseEntity<>	Bungkus response dengan status code yang bisa dikustomisasi

6. Database MySQL & JPA / ORM

6.1 application.properties

```
# Koneksi database spring.datasource.url=jdbc:mysql://localhost:3306/training_db
spring.datasource.username=root spring.datasource.password= # JPA/Hibernate
spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect # Port aplikasi (opsional,
default 8080) server.port=8080
```

6.2 Entity Lengkap

```
@Entity @Table(name = "products") @Data // Lombok: generate getter/setter/toString
@NoArgsConstructor // Lombok: constructor kosong @AllArgsConstructor // Lombok: constructor
semua field public class Product { @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id; @Column(nullable = false) private String name; @Column(nullable = false)
private Double price; @CreationTimestamp // Audit trail – waktu dibuat private
LocalDateTime createdAt; @UpdateTimestamp // Audit trail – waktu diupdate private
LocalDateTime updatedAt; private String createdBy; // Siapa yang buat (untuk audit) }
```

■ *@CreationTimestamp dan @UpdateTimestamp adalah cara mudah membuat audit trail di Spring Boot!*

6.3 Repository

```
@Repository public interface ProductRepository extends JpaRepository { // Custom query –
cari produk berdasarkan nama (mengandung kata kunci) List
findByNameContainingIgnoreCase(String keyword); // Custom query – cari produk yang harganya
di bawah nilai tertentu List findByPriceLessThan(Double price); // Dengan @Query JPQL
@Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max") List
findByPriceRange(@Param("min") Double min, @Param("max") Double max); }
```

Method JpaRepository	SQL Setara
save(entity)	INSERT or UPDATE
findAll()	SELECT * FROM table
findById(id)	SELECT * WHERE id = ?
deleteById(id)	DELETE WHERE id = ?
existsById(id)	SELECT COUNT(*) WHERE id = ?
count()	SELECT COUNT(*) FROM table

7. Service & DTO

7.1 DTO (Data Transfer Object)

DTO digunakan untuk memisahkan data yang ditampilkan ke client dari Entity database. Ini praktik yang baik agar struktur database tidak langsung terbuka.

```
@Data @NoArgsConstructor @AllArgsConstructor public class ProductDTO { private Long id;
@NotBlank(message = "Nama produk tidak boleh kosong") private String name;
@Positive(message = "Harga harus lebih dari 0") private Double price; }
```

7.2 Service dengan Validasi

```
@Service public class ProductService { @Autowired private ProductRepository repository;
public List getAll() { return repository.findAll().stream() .map(p -> new
ProductDTO(p.getId(), p.getName(), p.getPrice())) .collect(Collectors.toList()); } public
ProductDTO getById(Long id) { Product product = repository.findById(id) .orElseThrow(() ->
new ResourceNotFoundException("Produk tidak ditemukan, id: " + id)); return new
ProductDTO(product.getId(), product.getName(), product.getPrice()); } public ProductDTO
create(ProductDTO dto) { Product product = new Product(); product.setName(dto.getName());
product.setPrice(dto.getPrice()); return toDTO(repository.save(product)); } public void
delete(Long id) { if (!repository.existsById(id)) { throw new
ResourceNotFoundException("Produk tidak ditemukan, id: " + id); }
repository.deleteById(id); } private ProductDTO toDTO(Product p) { return new
ProductDTO(p.getId(), p.getName(), p.getPrice()); } }
```

8. Exception Handling

8.1 Custom Exception

```
// File: exception/ResourceNotFoundException.java public class ResourceNotFoundException extends RuntimeException { public ResourceNotFoundException(String message) { super(message); } }
```

8.2 Global Exception Handler

```
// File: exception/GlobalExceptionHandler.java @ControllerAdvice public class GlobalExceptionHandler { @ExceptionHandler(ResourceNotFoundException.class) public ResponseEntity> handleNotFound( ResourceNotFoundException ex) { Map error = new HashMap<>(); error.put("status", "404"); error.put("message", ex.getMessage()); return ResponseEntity.status(404).body(error); } @ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity> handleValidation( MethodArgumentNotValidException ex) { Map errors = new HashMap<>(); ex.getBindingResult().getFieldErrors() .forEach(e -> errors.put(e.getField(), e.getDefaultMessage())); return ResponseEntity.status(400).body(errors); } @ExceptionHandler(Exception.class) public ResponseEntity> handleGeneral(Exception ex) { Map error = new HashMap<>(); error.put("status", "500"); error.put("message", "Internal server error"); return ResponseEntity.status(500).body(error); } }
```

■ *@ControllerAdvice menangkap semua exception dari seluruh controller secara global!*

9. File Transfer (Upload & Download)

9.1 Konfigurasi di application.properties

```
spring.servlet.multipart.max-file-size=10MB spring.servlet.multipart.max-request-size=10MB file.upload-dir=./uploads
```

9.2 Controller Upload & Download

```
@RestController @RequestMapping("/files") public class FileController { @Value("${file.upload-dir}") private String uploadDir; // UPLOAD @PostMapping("/upload") public ResponseEntity upload(@RequestParam MultipartFile file) { try { String filename = System.currentTimeMillis() + "_" + file.getOriginalFilename(); Path path = Paths.get(uploadDir).resolve(filename); Files.copy(file.getInputStream(), path, StandardCopyOption.REPLACE_EXISTING); return ResponseEntity.ok("File berhasil diupload: " + filename); } catch (IOException e) { return ResponseEntity.status(500).body("Gagal upload file"); } } // DOWNLOAD @GetMapping("/download/{filename}") public ResponseEntity download(@PathVariable String filename) { try { Path path = Paths.get(uploadDir).resolve(filename); Resource resource = new UrlResource(path.toUri()); if (!resource.exists()) throw new ResourceNotFoundException("File tidak ditemukan"); return ResponseEntity.ok() .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename="" + filename + """) .body(resource); } catch (Exception e) { throw new ResourceNotFoundException("File tidak dapat diunduh"); } } }
```

■ *Selalu validasi tipe file (cek ekstensi & MIME type) untuk keamanan!*

10. Security (Spring Security + JWT)

Spring Security digunakan untuk autentikasi (siapa kamu?) dan otorisasi (kamu boleh akses apa?).

10.1 Dependency yang Dibutuhkan

```
org.springframework.boot spring-boot-starter-security io.jsonwebtoken jjwt-api 0.11.5
```

10.2 Konsep JWT (JSON Web Token)

Bagian JWT	Isi	Contoh
Header	Tipe token & algoritma	{ alg: HS256, typ: JWT }
Payload	Data user (claims)	{ sub: user, role: ADMIN, exp: ... }
Signature	Verifikasi integritas token	HMACSHA256(base64(header)+'.'+base64(payload), secret)

10.3 Alur Autentikasi JWT

```
1. User POST /auth/login → { username, password } 2. Server validasi → buat JWT token → kirim ke client 3. Client simpan token 4. Setiap request berikutnya: kirim header: Authorization: Bearer 5. Server verifikasi token → proses request
```

10.4 Security Config Dasar

```
@Configuration @EnableWebSecurity public class SecurityConfig { @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http .csrf().disable() .sessionManagement() .sessionCreationPolicy(SessionCreationPolicy.STATELESS) .and() .authorizeHttpRequests() .requestMatchers("/auth/**").permitAll() // Login bebas akses .requestMatchers("/admin/**").hasRole("ADMIN") .anyRequest().authenticated(); return http.build(); } @Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); } }
```

■ *JANGAN PERNAH menyimpan password plaintext! Selalu gunakan BCryptPasswordEncoder.*

11. Audit Trail

Audit trail adalah pencatatan otomatis siapa yang membuat/mengubah data dan kapan waktunya. Ini sangat penting untuk aplikasi enterprise.

11.1 Cara Otomatis dengan @EnableJpaAuditing

```
// 1. Aktifkan di main class @SpringBootApplication @EnableJpaAuditing(auditorAwareRef = "auditorProvider") public class DemoApplication { ... } // 2. Buat AuditorAware untuk ambil user yang login @Component("auditorProvider") public class AuditorAwareImpl implements AuditorAware { @Override public Optional getCurrentAuditor() { // Ambil username dari SecurityContext Authentication auth = SecurityContextHolder.getContext().getAuthentication(); if (auth == null || !auth.isAuthenticated()) return Optional.of("system"); return Optional.of(auth.getName()); } } // 3. Tambahkan di Entity @Entity @EntityListeners(AuditingEntityListener.class) public class Product { @CreatedDate private LocalDateTime createdAt; @LastModifiedDate private LocalDateTime updatedAt; @CreatedBy private String createdBy; @LastModifiedBy private String updatedBy; }
```

■ *Dengan setup ini, setiap save/update otomatis mencatat waktu dan user yang melakukan perubahan.*

12. Testing dengan JUnit

Testing memastikan kode kita berjalan sesuai harapan. Spring Boot mendukung unit test (test per komponen) dan integration test.

12.1 Unit Test – Service

```
@ExtendWith(MockitoExtension.class) class ProductServiceTest { @Mock private
ProductRepository repository; @InjectMocks private ProductService service; @Test void
testGetAll_ShouldReturnList() { // Arrange (siapkan data dummy) List dummyList = List.of(
new Product(1L, "Keyboard", 350000.0), new Product(2L, "Mouse", 150000.0) );
when(repository.findAll()).thenReturn(dummyList); // Act (jalankan method yang ditest) List
result = service.getAll(); // Assert (cek hasilnya) assertEquals(2, result.size());
assertEquals("Keyboard", result.get(0).getName()); } @Test void
testGetById_NotFound_ShouldThrowException() {
when(repository.findById(99L)).thenReturn(Optional.empty());
assertThrows(ResourceNotFoundException.class, () -> service.getById(99L)); } }
```

12.2 Integration Test – Controller

```
@SpringBootTest @AutoConfigureMockMvc class ProductControllerTest { @Autowired private
MockMvc mockMvc; @Test void testGetAllProducts() throws Exception {
mockMvc.perform(get("/products")) .andExpect(status().isOk())
.andExpect(jsonPath("$.length()").value(greaterThanOrEqualTo(0))); } @Test void
testCreateProduct() throws Exception { String json = "{"name":"Keyboard","price":350000}";
mockMvc.perform(post("/products") .contentType(MediaType.APPLICATION_JSON) .content(json))
.andExpect(status().isCreated()) .andExpect(jsonPath("$.name").value("Keyboard")); } }
```

■ *@Mock = objek tiruan, @InjectMocks = inject mock ke class yang ditest, when() = definisikan perilaku mock.*

13. Pagination & Sorting (BONUS)

Pagination penting agar aplikasi tidak mengirim semua data sekaligus ke client, yang bisa memperlambat performa.

```
// Controller @GetMapping public Page getAll( @RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "10") int size, @RequestParam(defaultValue = "id") String sortBy) { Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy)); return service.getAll(pageable); } // Service public Page getAll(Pageable pageable) { return repository.findAll(pageable).map(this::toDTO); }
```

```
// Contoh request: GET /products?page=0&size;=5&sortBy;=name // Contoh response: { "content": [...], "totalElements": 50, "totalPages": 10, "number": 0, "size": 5 }
```

14. Studi Kasus – CRUD Product Lengkap

Step-by-Step Panduan

STEP 1: Setup Project	Buka IntelliJ → New Project → Spring Initializr Dependency: Spring Web, Spring Data JPA, MySQL Driver, Lombok, Spring Security, Validation
STEP 2: Buat Database	Buka DBeaver → jalankan: CREATE DATABASE training_db;
STEP 3: Isi application.properties	Isi konfigurasi datasource & JPA seperti di Bab 6.1
STEP 4: Buat Entity	Buat class Product.java di folder entity/ dengan anotasi @Entity, @Id, @CreationTimestamp, dll
STEP 5: Buat Repository	Buat interface ProductRepository extends JpaRepository
STEP 6: Buat DTO	Buat class ProductDTO.java di folder dto/ dengan validasi @NotBlank, @Positive
STEP 7: Buat Service	Buat class ProductService.java di folder service/ lengkap dengan CRUD + exception handling
STEP 8: Buat Controller	Buat class ProductController.java di folder controller/ dengan GET, POST, PUT, DELETE
STEP 9: Buat Exception Handler	Buat class GlobalExceptionHandler.java di folder exception/
STEP 10: Jalankan & Test	Klik Run → buka Postman → test semua endpoint

15. Checklist Siap Pelatihan

- [ ] Java 17 terinstall (cek: java -version)
- [ ] Maven berjalan (cek: mvn -v)
- [ ] IntelliJ IDEA bisa dibuka dan berjalan normal
- [ ] Laragon / XAMPP – MySQL berjalan
- [ ] DBeaver bisa connect ke MySQL local
- [ ] Postman terinstall dan bisa kirim request
- [ ] Bisa membuat project Spring Boot baru via Spring Initializr
- [ ] Bisa menjalankan aplikasi di localhost:8080
- [ ] Paham konsep GET POST PUT DELETE
- [ ] Bisa membuat Entity sederhana dengan JPA
- [ ] Paham konsep Class, Object, Inheritance di Java OOP
- [ ] Sudah baca buku saku ini minimal sekali

■ *Jika semua checklist terpenuhi, kamu siap mengikuti pelatihan dan tidak akan kesulitan mengerjakan studi kasus!*